

Пројектовање и анализа алгоритама

Александар Госпавић 19095

Пројекат 2

Проблем 2, Insertion - Heap - Bucket

Програм за сваку величину низа генерише нови низ са насумичним бројевима између 0 и 10000 (помоћу класе *RNG*) и њега користи за решавање проблема помоћу сва три алгорита. Сви резултати се приказују на конзолу.

Како је *Insertion Sort* временске сложености $O(n^2)$, за низове са 1M и 10M елемената сам прекинуо ивршење и штампао *N/A (Not available)* јер за комплетно извршење програма је потребно више сати.

Insertion Sort не захтева додатни меморијски простор.

Heap Sort захтева меморијски простор $O(\log n)$ због рекурзивних позива, а иначе не заузима никакве додатне ресурсе. Међутим, за израчунавање потребне меморије за сваки алгоритам, користио сам системске методе које враћају стање Garbage Collector-а. Тако сам измерио стање пре и после позива одређеног сорт алгорита и једноставним одузимањем штампао резултат. Због тога, за *Heap Sort* се приказује 0 бајта јер рекурзивни позиви нису урачунати.

Bucket Sort је убедљиво најбржи алгоритам јер му је временска сложеност $O(n)$, али користи највише простора што ће бити и јасно видљиво у резултатима.

Закључак:

- Може се приметити да Insertion сорт, иако најспорији алгоритам, за мале низове не даје ништа горе резултате од *Heap Sort-a* и *Bucket Sort-a*, чак за 100 елемената видимо да је и најбржи.
- За низове са више од 10000 елемената, перформансе *Insertion Sort-a* значајно опадају.
- *Bucket Sort* је за нијансу био спорији код низа са 100 елемената, међутим за сваки други низ показује најбоље резултате, али и највише утрошене меморије.

Апликација је писана у програмском језику C# (.NET 8.0)

Резултати у овом извештају су када се програм покрене у *Release* моду.

Следе детаљнији резултати.

```

Niz od 100 elemenata...
N: 100, K: 20

Minimalna cena      : 20123
Utrosak memorije    : 0 bytes
Koriscen InsertionSort : 0.0006744 s

Minimalna cena      : 20123
Utrosak memorije    : 0 bytes
Koriscen HeapSort   : 0.0007088 s

Minimalna cena      : 20123
Utrosak memorije    : 8224 bytes
Koriscen BucketSort : 0.0077094 s

```

```

-----
Niz od 1000 elemenata...
N: 1000, K: 200

Minimalna cena      : 20642
Utrosak memorije    : 0 bytes
Koriscen InsertionSort : 0.0024775 s

Minimalna cena      : 20642
Utrosak memorije    : 0 bytes
Koriscen HeapSort   : 0.0004926 s

Minimalna cena      : 20642
Utrosak memorije    : 67616 bytes
Koriscen BucketSort : 0.0002778 s

```

```

-----
Niz od 10000 elemenata...
N: 10000, K: 2000

Minimalna cena      : 20010
Utrosak memorije    : 0 bytes
Koriscen InsertionSort : 0.0559321 s

Minimalna cena      : 20010
Utrosak memorije    : 0 bytes
Koriscen HeapSort   : 0.0047952 s

Minimalna cena      : 20010
Utrosak memorije    : 659136 bytes
Koriscen BucketSort : 0.0026314 s

```

```

Niz od 100000 elemenata...
N: 100000, K: 20000

Minimalna cena      : 19989
Utrosak memorije    : 0 bytes
Koriscen InsertionSort : 6.9842163 s

Minimalna cena      : 19989
Utrosak memorije    : 0 bytes
Koriscen HeapSort   : 0.0563853 s

Minimalna cena      : 19989
Utrosak memorije    : 4675480 bytes
Koriscen BucketSort : 0.0409256 s

```

```

-----
Niz od 1000000 elemenata...
N: 1000000, K: 200000

Minimalna cena      : N/A
Utrosak memorije    : N/A bytes
Koriscen InsertionSort : N/A s

Minimalna cena      : 20020
Utrosak memorije    : 0 bytes
Koriscen HeapSort   : 0.6282929 s

Minimalna cena      : 20020
Utrosak memorije    : 39929928 bytes
Koriscen BucketSort : 0.2345996 s

```

```

-----
Niz od 100000000 elemenata...
N: 100000000, K: 20000000

Minimalna cena      : N/A
Utrosak memorije    : N/A bytes
Koriscen InsertionSort : N/A s

Minimalna cena      : 19997
Utrosak memorije    : 0 bytes
Koriscen HeapSort   : 3.9227537 s

Minimalna cena      : 19997
Utrosak memorije    : 408149496 bytes
Koriscen BucketSort : 3.0998874 s

```